A/B Testing Toolkit

Planning, checking, analysing and deciding on online experiments

Final Project Report

Mohamed Moin Peer

MSc Data Analytics, Queen's University Belfast

Portfolio project | 2026

Abstract

This project builds a Python toolkit for the full life cycle of an online experiment. It covers power analysis and duration planning; sample-ratio-mismatch (SRM) checks; frequentist, bootstrap and Bayesian comparisons; CUPED variance reduction; multiple-testing correction; and group-sequential monitoring. Explicit decision rules turn each analysis into an auditable decision. The toolkit was applied to five synthetic experiments for a fictional online retailer, each generated with a known true effect, and all five decisions matched the truth. Detecting a 10% relative lift on a 3.2% conversion rate required 49,774 visitors per group (14 days). An apparently strong result (+9.6%) was correctly withheld because of an SRM ($p = 1.5 \times 10^{-8}$). A segment-level false positive ($p = 0.02$) disappeared after Holm correction ($p = 0.08$). CUPED reduced the variance of 30-day spend by 64.8% and moved the estimate from £0.92 to £0.52 (true £0.60). Simulation showed that daily peeking raises the false-positive rate to 21.6%; group-sequential boundaries restored 4.8%.

1. Introduction and aims

Controlled experiments are the standard way for digital businesses to decide whether a change improves an outcome. Their value depends on sound practice. Tests must be large enough. Their randomisation must be intact. They must not be stopped early on a favourable reading, and results must not be mined across segments. This project aimed to:

- implement planning tools for sample size, minimum detectable effect, power and duration;
- implement and validate a set of analysis methods against reference libraries and by simulation;
- encode explicit, pre-registered decision rules and generate a report per experiment;
- demonstrate, with experiments of known outcome, how common failures arise and how the toolkit prevents them;
- provide a browser calculator that applies the same methods without installation.

2. Data

All data is synthetic, produced by src/ab_toolkit/generate_data.py with a fixed seed, for a fictional online homeware retailer. Four website experiments contain about 100,000 visitors each. Each visitor record holds the group, device, visitor type, conversion, order value and whether the order was returned. Baseline conversion varies by device, visitor type and weekday. Order values follow a log-normal distribution. An email experiment covers 40,000 loyalty members, with spend in the 30 days before and after the email and whether it was opened.

Experiment	Built-in truth
One-page checkout redesign	+10% relative conversion; no effect on returns
Free delivery from £30 (was £40)	+6% conversion, −5% order value
Recommendations widget	No effect
Show price per unit	+3% conversion; a tracking bug drops 6% of non-buying mobile treatment visitors

Experiment	Built-in truth
Personalised email subject line	+£0.60 on 30-day spend (mean about £22)

3. Methods

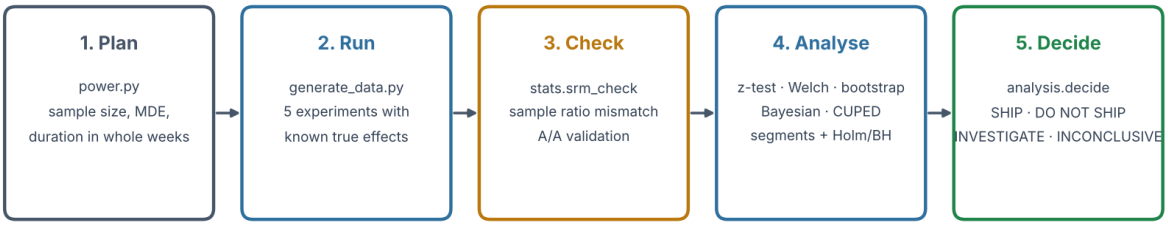


Figure 1. Toolkit workflow.

3.1 Planning

The per-group sample size for comparing proportions p_1 and p_2 uses the normal approximation with unpooled variances, $n = (z_{1-\alpha/2} + z_{1-\beta})^2 [p_1(1 - p_1) + p_2(1 - p_2)] / (p_2 - p_1)^2$. Durations are rounded up to whole weeks so that each weekday is equally represented. The minimum detectable effect for a given sample is found by bisection. Results agree with statsmodels' NormalIndPower to within 2%.

3.2 Health checks

The SRM check compares the observed split with the designed split using a chi-square goodness-of-fit test, with a strict threshold of $p < 0.001$, as is common practice. The tests themselves were validated by simulating 2,000 A/A experiments.

3.3 Analysis

- Conversion rates: two-proportion z-test (pooled standard error for the test, unpooled for the interval).
- Means: Welch's t-test; skewed revenue per visitor: percentile bootstrap (2,000 resamples).
- Bayesian: Beta(1,1)–Binomial posteriors, giving P(treatment better), the relative-lift credible interval and the expected loss of each choice.
- CUPED: $Y' = Y - \theta(X - \bar{X})$ with $\theta = \mathrm{cov}(Y, X) / \mathrm{var}(X)$ from a pre-randomisation covariate X, estimated on pooled data.
- Segments: four device × visitor-type cells with Holm and Benjamini–Hochberg adjustments.
- Sequential: O'Brien–Fleming-shaped boundaries $z_k = C\sqrt{(K/k)}$, with C calibrated by simulation for 14 equally spaced looks at an overall two-sided α of 0.05.

3.4 Decision rules

Order	Condition	Decision
1	SRM detected (p < 0.001)	INVESTIGATE
2	A guardrail metric significantly worse	DO NOT SHIP
3	Primary metric significantly better	SHIP
4	Primary metric significantly worse	DO NOT SHIP
5a	Not significant; CI excludes the planned effect	NO MEANINGFUL EFFECT
5b	Not significant; CI includes the planned effect	INCONCLUSIVE

4. Results

Experiment	Primary change	p-value	SRM p	Decision
One-page checkout redesign	+9.4%	0.00732	0.1	SHIP
Free delivery from £30 (was £40)	+1.6%	0.643	0.89	NO MEANINGFUL EFFECT
'You may also like' widget	+5.5%	0.121	0.31	INCONCLUSIVE
Show price per unit	+9.6%	0.00695	1.5e-08	INVESTIGATE
Personalised email subject line	+2.3%	0.000324	0.56	SHIP

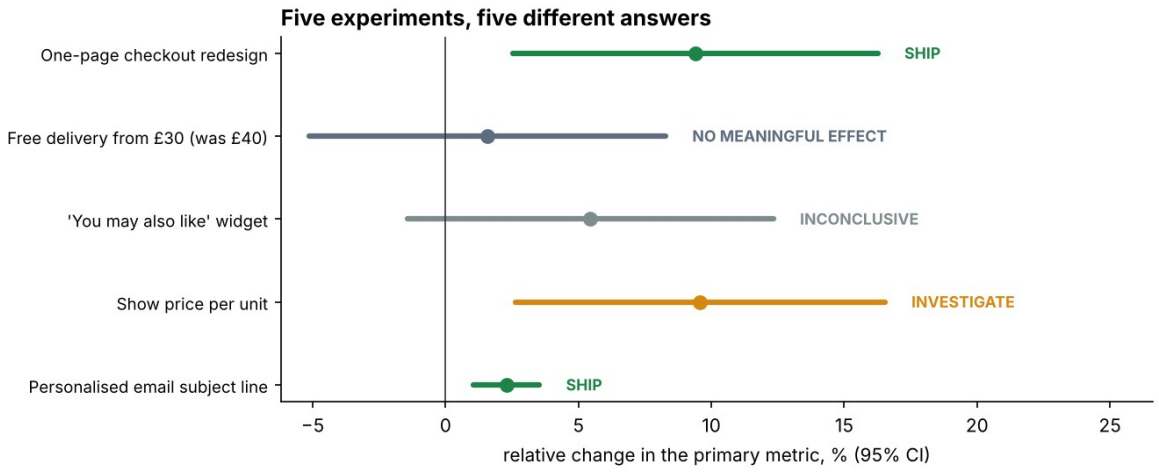


Figure 2. Relative change in each primary metric with 95% confidence intervals.

4.1 Checkout redesign

Conversion rose by 9.4% (p = 0.0073), with P(better) = 99.6% and no deterioration in the return rate. The estimate is close to the true 10% and the decision is to ship.

4.2 Free delivery threshold

Conversion changed by +1.6% (p = 0.64). The interval excluded the planned 10% lift, giving NO MEANINGFUL EFFECT. Revenue per visitor fell by 2.3% (bootstrap p = 0.60) and order value by 3.8%, consistent with the built-in trade-off. The true 6% conversion lift was below the effect the test was designed to detect, which shows the cost of a test sized for larger effects.

4.3 Recommendations widget and multiple testing

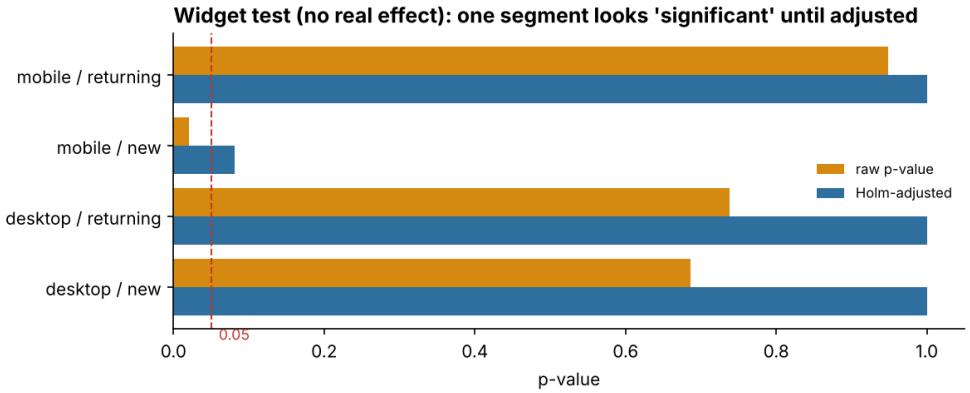


Figure 3. Raw and Holm-adjusted p-values by segment for the widget test.

The overall result was not significant (p = 0.12). One of four segments (mobile, new visitors) showed p = 0.02. That is significant in isolation but not after Holm adjustment (p = 0.08). Because the true effect is zero, this is a false discovery that the adjustment correctly removes.

4.4 Unit price display and SRM

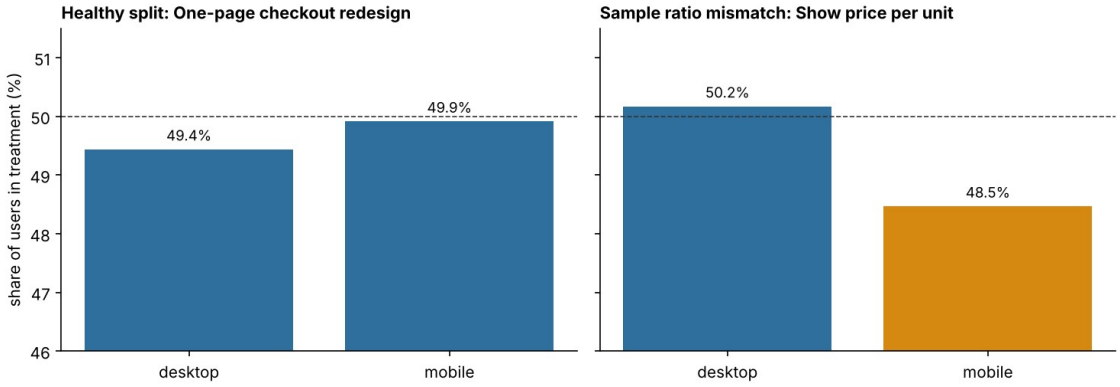


Figure 4. Share of users in treatment by device for a healthy and a broken test.

The treatment group received 49.1% of users against a designed 50% (p = 1.5e-08). The deficit was concentrated in mobile traffic. Because the lost users were non-buyers, the observed lift of 9.6% overstates the true 3%. The rules stop at the health check, and the decision is to investigate and re-run.

4.5 Email experiment and CUPED

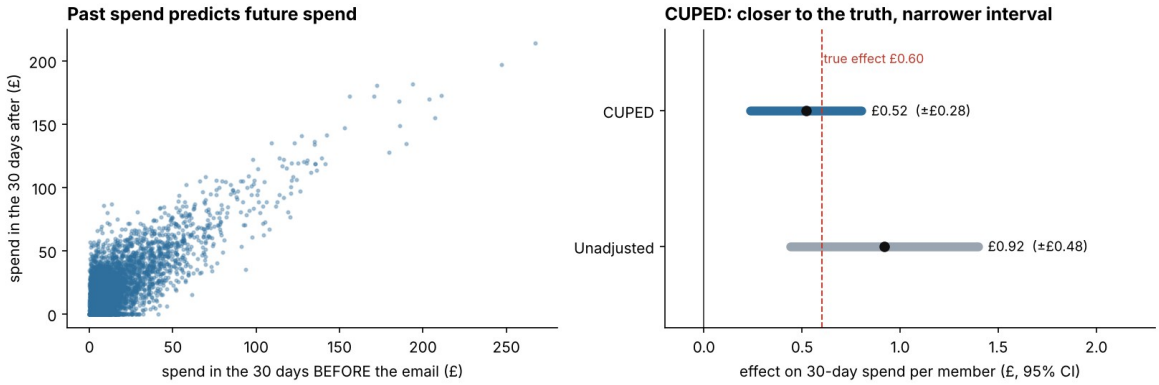


Figure 5. Pre-period and post-period spend, and the unadjusted and CUPED estimates.

Pre-period spend explained most of the variation in 30-day spend: CUPED removed 64.8% of the variance and narrowed the 95% interval by 40.7%. By chance, the treatment group had spent £0.47 more before the email (p = 0.04). This inflated the unadjusted estimate to £0.92 (95% CI £0.44–£1.40). CUPED gave £0.52 (£0.24–£0.80), close to the true £0.60. Both lead to SHIP, but only the adjusted estimate is suitable for forecasting the revenue impact.

4.6 Peeking and sequential testing

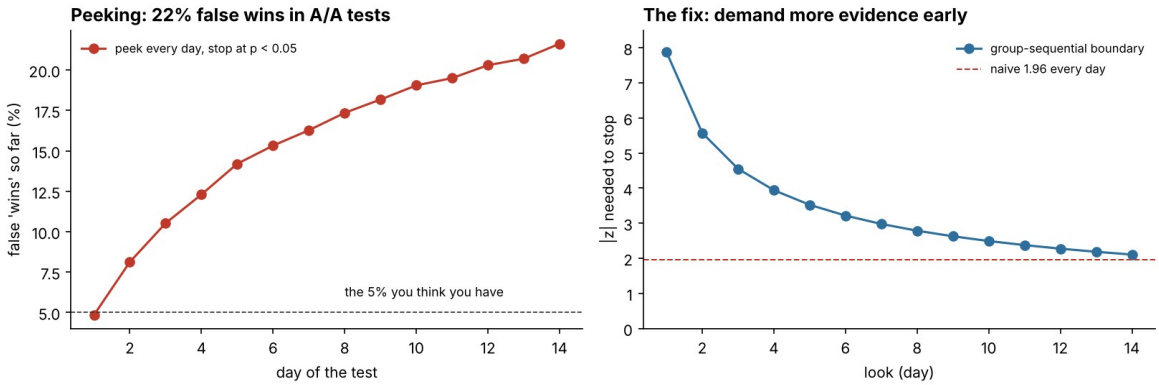


Figure 6. Cumulative false-positive rate under daily peeking, and the group-sequential boundaries.

Across 4,000 simulated A/A tests with 14 daily looks, stopping at the first p < 0.05 produced false positives in 21.6% of tests, against 4.9% for a single final analysis. Group-sequential boundaries (|z| ≥ 7.9 on day 1, falling to 2.1 on day 14) gave 4.8%. With a true 10% lift, they detected the effect in 77.4% of tests, against 79.1% for a single look, and stopped on day 10.7 on average.

4.7 Bayesian analysis and validation

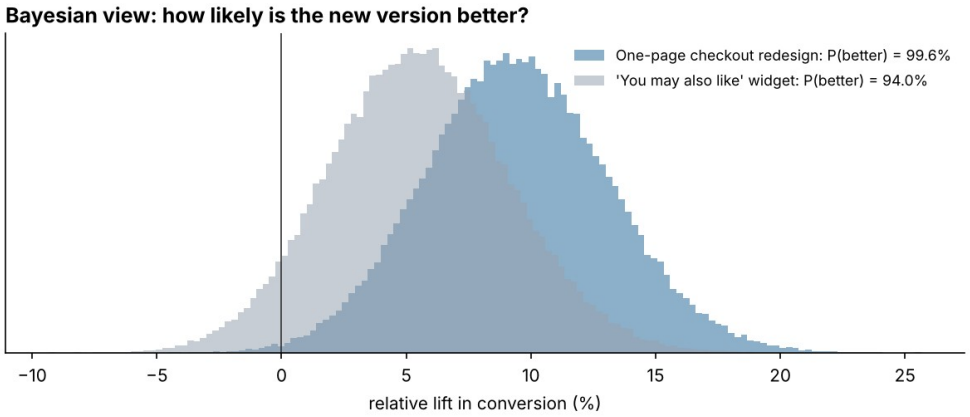


Figure 7. Posterior distributions of relative lift.

P(better) was 99.6% for the checkout. It was also 94.0% for the widget, which has no true effect. A threshold rule on P(better) alone would therefore have shipped the widget. This is why the decision rules use intervals and the planned effect size. In 2,000 simulated A/A tests the z-test gave 5.1% false positives, with uniformly distributed p-values.

5. Implementation and testing

The toolkit is a Python package (power, stats, sequential, analysis, reporting) with a command-line interface for data generation, analysis and test planning. It writes a Markdown decision report for each experiment, nine figures, and CSV and JSON results. Twenty-four pytest tests check the sample size against statsmodels; the z-test, Welch test and p-value adjustments against statsmodels and SciPy; the bootstrap interval coverage, CUPED unbiasedness and variance reduction, and SRM detection; the decision rules against the known truths; the A/A false-positive rate; and the peeking simulation. A single-file browser calculator (web/index.html) implements the sample-size, significance and SRM tools with identical formulas.

6. Limitations and future work

- The data is synthetic and cleaner than real experiments, with no novelty effects, interference or seasonality shifts during a test.
- Metrics are user-level; ratio metrics measured per session would need the delta method.
- Sequential boundaries assume equally spaced looks; alpha-spending functions would allow flexible timing.
- The Bayesian analysis uses a flat prior; informative priors from past experiments could be used.
- Future work: delta-method ratio metrics, alpha spending, model-based heterogeneous treatment effects, and running the toolkit on an event table through SQL.

7. Conclusion

The toolkit reached the correct decision in all five experiments. That includes the cases where a naive reading would have been wrong: a broken traffic split that produced the most impressive lift, and a segment result that was pure noise. The results support a simple practice. Plan the effect size and duration in advance, check the split before reading any metric, adjust for multiple comparisons, use pre-period data where available, and never stop early without a sequential design.

Appendix: how to run

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
python -m ab_toolkit all
python -m ab_toolkit plan --baseline 0.032 --mde 0.10 --daily 7143
pytest
```